Group 3

Team Members

- Ketan Babar
- Sarvesh Changan
- Khushi Jadhav

1) Project Title

TicketShield

Secure Hybrid Event Ticketing Platform with DevSecOps Pipeline

2) The Scenario

A large event company manages ticket sales for high-demand concerts, sports matches, and festivals. While daily traffic remains stable, ticket launches trigger massive request spikes within seconds, often resulting in API failures, payment issues, and inventory mismatches.

To address scalability challenges, the company is transitioning to a hybrid architecture where:

- Public-facing burst workloads are handled using AWS Lambda
- Core business services such as inventory, order processing, and authentication are deployed on Kubernetes for better control and service isolation

However, recent security concerns such as credential leakage, dependency vulnerabilities, and unauthorized API access have highlighted the need for a **secure DevSecOps pipeline**.

The client provides a **pre-built, intentionally insecure backend codebase** containing:

- Hardcoded secrets
- Vulnerable dependencies
- Basic Docker and Terraform configurations

The objective of this project is **not to develop the application**, but to design and implement a **secure DevSecOps pipeline** that validates, secures, provisions, and deploys the system automatically.
The pipeline must ensure:

- No insecure code is deployed
- Infrastructure is provisioned using Infrastructure as Code
- Deployments are reliable, secure, and reversible

3) Technical Requirements

A) Source Control

Use Git with GitHub or GitLab

- Follow GitFlow branching strategy (feature, develop, release, hotfix)
- Enforce protected main branch
- Require pull request reviews before merging
- Enable commit signing for integrity
- Integrate pre-commit checks for basic security validation

B) CI/CD Tool (DevSecOps Pipeline)

Use GitHub Actions or Jenkins

Pipeline Stages:

- Code Checkout
- Build and Dependency Installation
- Unit and Integration Testing
- Code Quality Analysis
- Security and Vulnerability Scanning
- Docker Image Build
- Infrastructure Provisioning (Terraform)
- Deployment (Conditional)
- Rollback Mechanism (on failure)

Deployment Strategy:

- Blue-Green or Canary deployment
- Automatic rollback on failure or security violation

C) Infrastructure

Use Terraform for provisioning

Hybrid Architecture:

- **Serverless Layer**
    - AWS Lambda for handling high traffic bursts
    - API Gateway for request routing
- **Kubernetes Layer**
    - Core services deployed using containers
    - Local Kubernetes setup (Minikube/Kind) allowed for student implementation
- **Containerization**
  Use Docker
- **Security & Access**
    - IAM roles with least privilege
    - Secret management using HashiCorp Vault

D) DevSecOps & Vulnerability Scanning

Security must be integrated at every stage of the pipeline.

Tools (Free / Open Source):

- Vulnerability Scanning → Trivy
- Secret Detection → Gitleaks
- Code Quality Analysis → SonarQube

Security Checks:

- Detect hardcoded secrets in source code
- Scan dependencies for known vulnerabilities
- Scan Docker images for OS/package vulnerabilities
- Enforce secure coding standards

Security Gate Logic (Mandatory):

Deployment must proceed **only if**:

- No hardcoded secrets are detected
- No HIGH or CRITICAL vulnerabilities are found
- Code quality passes defined thresholds

If any condition fails:
Pipeline must **fail immediately and block deployment**

E) Provided Codebase

The client will provide a sample backend repository containing:

- A simple backend service (Node.js / Java)
- Intentionally vulnerable dependencies
- Hardcoded secrets
- Dockerfile
- Basic Terraform configuration

The contractor team is required to **implement the DevSecOps pipeline on this codebase only**, without developing additional application features.

F) Monitoring & Observability

Track the following metrics:

Performance Metrics:

- API latency
- Lambda invocation errors
- Kubernetes pod restarts

Security Metrics:

- Unauthorized access attempts
- Secret detection logs
- Vulnerability scan results

Pipeline Metrics:

- Build success/failure rate
- Number of deployments blocked due to security issues

Business Simulation Metrics:

- Ticket booking success rate
- Overselling prevention checks (simulated logic)

Tools: Prometheus , Grafana

4) Success Criteria

The project is considered successful if:

- A code commit automatically triggers the DevSecOps pipeline
- Security scanning detects and blocks vulnerable or insecure code
- No hardcoded secrets are present in the repository
- Infrastructure is fully provisioned using Terraform
- Hybrid deployment (Serverless + Kubernetes) is achieved
- The pipeline enforces all security gates before deployment
- Automatic rollback is triggered in case of failure
- The system maintains consistency and prevents simulated overselling issues

Note

This project focuses on implementing a **secure, automated DevSecOps pipeline** using a provided insecure codebase, ensuring real-world security practices such as vulnerability detection, secret management, and controlled deployment are enforced effectively.

Repo Link
https://github.com/Sarvesh-Changan/Event-booking